この記事は最終更新日から5年以上が経過しています。

@shin_hayata

Elasticsearchを日本語で使う設定のまとめ

Elasticsearch

最終更新日 2017年03月20日 投稿日 2017年03月20日

すっかり春めいてきましたねぇ。もう少しで🌸の季節です。

AWSでElasticsearch（5.1）が使える！

2017年1月30日から、AWSのElasticsearchで5.1.1が使えるようになりました（AWSのお知らせ）とっても嬉しいこのアップデート、APIがすぐに試せるKibanaの開発consoleもついてきて、とても使いやすくなりました。

さてさて、しかし、日本語を検索するシステムで使う場合は、Elasticsearchに日本語の設定をしないと使えないですね。日本語の文字の細かい処理...というと、Windows XPからWindows 7に切り替わるときに、表示される字体が変わってしまったいわゆるJIS2004問題を思い出しますが...もう一昔前の話なんでしょうか...👴
『辻』（つじ）という文字が、一点シンニョウから二点シンニョウになった、アレですね。

日本語を使うときの設定は？

AWSのElasticsearchは、日本語プラグインインストール済みです。今日はこのプラグ

1. ICU
2. Kuromoji

この２つは共によく知られているライブラリのようですので、ご存知の方もいらっしゃると思いますが、ICUはInternational Components for Unicodeの略で、Unicodeの国際化サポートをしてくれるライブラリ、Kuromojiは日本語の形態素解析ライブラリです。

Elasticsearchは、全文検索ライブラリLuceneを動かしています。全文検索が上手く動くためには、検索用の転置インデックスを作るときに、日本語のインデックスが（いわゆる分かち書きによって単語に区切られ、それらの単語の表記が正規化されることで）上手くヒットするようになる、ということですね。

Elasticsearch 5.1のデフォルト設定は？

日本語でAnalyzeするフィールドにKuromoji analyzerを設定すれば、大体は良い感じに検索フィールドができあがりました😄

AWSのElasticsearchではプリインストールされているので、インストールは特に必要ありません。ローカルで動かす場合は、ガイドに記載されたとおりコマンドでインストールしました。

console(linux)

```
sudo bin/elasticsearch-plugin install analysis-kuromoji
```

Analyzerの設定の仕方は公式ガイドのこのページに記載のとおりで、Indexを作成するクエリの中で、Analyzerの名前に"kuromoji"と指定します。

kibana(devtool)

```
PUT my_index
{
  "mappings": {
    "my_type": {
      "properties": {
        "field_name": {
          "type":     "text",
```

271 250 •••

```
      }
    }
  }
}
```

動かしてみると良い感じで、（特殊な文字が入ることを想定しなくても良いという意味で）普通に使うシステムではこれで十分に使えそうでした。

ここまでお読みくださってありがとうございます！以降は細かいです！

しかし、この初老プログラマ👴が社用システムで使う環境は、古いマニュアル📠とかも多くあるので、文字の処理は念入りにしておく必要がありそうです😣苦痛

このKuromoji analyzerに含まれるtokenizerとtoken filtersの一覧を細かく見ると、ICUプラグインの機能は入ってないようです。

モジュール	機能	使いたい？（後述）
kuromoji_tokenizer	トークン化	Yes
kuromoji_baseform token filter	原形化	Yes
kuromoji_part_of_speech token filter	品詞タグ付	Yes
cjk_width token filter	全角半角変換	No(※1)
ja_stop token filter	ストップワードの除去	Yes
kuromoji_stemmer token filter	長音除去	Yes
lowercase token filter	小文字化	No(*1)

(※1) ICUプラグインで代替・置換できるため

Kuromoji analyzerに含まれているCJK Width Token Filterのページには、この記載があります。

271 250 •••

> normalization. See the analysis-icu plugin for full normalization support.

ICUプラグインを入れると、完全な正規化がサポートされていて、このCJK Width Token Filterの機能は、ICUに含まれているよ！ということなので、今回はICUプラグインを入れて、置き換えてみます。

ちなみに、CJK Width Token Filterの機能は、

- 全角ASCII文字を、等価な半角文字に変換する
- 半角ｶﾀｶﾅを、全角カタカナに変換する

という、全角←→半角変換の機能のみです。

用意されているプラグインの機能一覧

ICUプラグインを入れるなら、全部調べてみよう！ということでElasticsearchの公式ガイドを見ると、

Japanese (kuromoji) Analysis Plugin

Plugin	機能	使いたい？
kuromoji_iteration_mark character filter	踊り字の正規化	Yes
kuromoji_tokenizer	トークン化	Yes
kuromoji_baseform token filter	原形化	Yes
kuromoji_part_of_speech token filter	不要な品詞の除去	Yes
kuromoji_readingform token filter	読み仮名付与	Yes
kuromoji_stemmer token filter	長音の除去	Yes
ja_stop token filter	ストップワードの除去	Yes
kuromoji_number token filter	漢数字の半角数字化	Yes

ICU Analysis Plugin

271 250 •••

Plugin	機能	使いたい？
ICU Normalization Character Filter	文字の正規化	Yes
ICU Tokenizer	トークン化	No(※1)
ICU Normalization Token Filter	文字の正規化	No(※2)
ICU Folding Token Filter	長音統一化	No(※3)
ICU Collation Token Filter	辞書順ソート	No(※4)
ICU Transform Token Filter	音訳など	No(※5)

だけ機能があるようですので、ガイトの目次順に確認したいと思います。

(※1) Kuromojiを使うため
(※2) Character Filterを使うため
(※3) 今回検索に使用しない言葉（品詞）が対象のため
(※4) スコア順にソートするため、辞書順にソートしないため
(※5) 読み音では検索しないため
これらの理由は後述の確認の結果です。

Kuromojiプラグイン機能のそれぞれ

kuromoji_iteration_mark character filter

踊り字を、正規化してくれます。

例）時々→時時

文学的な表現も現代的な表現になりますね。

例）こゝろ→こころ
例）学問のすゝめ　→　学問のすすめ

kuromoji_tokenizer

（デフォルトの）searchモードで使います。

271 250 •••

kuromoji_baseform token filter

活用により表記が変わっている言葉を、原形に統一してくれます。

例）飲み　→　飲む

kuromoji_part_of_speech token filter

検索に有用でない、助詞などを品詞タグに基いて削除してくれます。

例）寿司がおいしいね　→　「寿司」「おいしい」だけ残して、「が」と「ね」を削除

kuromoji_readingform token filter

読み仮名で検索をヒットできるようにしてくれます。

例）寿司　→　「寿司」「スシ」の両方でヒット

これにより、送りかなの表記揺れも吸収できそうです。

実例）行う　→　オコナウ
　　行なう → オコナウ

＃実際に_analyzeコマンドでやってみるとわかりますが、実は送りかなを変えると、読み方を誤ってしまうことが多々あるので、読み仮名だけで検索をかけるのは難しいかもしれません。

実例）明るい　→　アカルイ
　　明かるい　→　メイ・カルイ（そもそも別のtokenになってしまいました…）

「スシ」の例もそうでしたが、感じの他に、わざとカタカナで書いたりする言葉も、検索できるようになりそうです。

実例）幸せ　→　シアワセ

＃しかし、ひらがなの「しあわせ」は、「仕合わせ」と類似の解釈なのか、「し・あわ・せ」でバラバラのTokenに分解されてしまいました。

そして、根本的に、読み方が同じで別の漢字（同音異義語と申しますか…）が全て

271　250　•••

例）（ペットを）飼う　→　カウ
　　（ペットを）買う　→　カウ

あちらを立てればこちらが立たず…これが検索システムの適合率と再現率の話なんでしょうか…とほほ👴

ちなみに、読み仮名は、カタカナ（スシ）とローマ字（sushi）の２種類から選べますが、ガイドのとおり、カタカナで設定するほうがanalyzeでは分かりやすいと思いました。

kuromoji_stemmer token filter

単語の最後の長音を削除して統一してくれます。
例）プリンター　→　プリンタ

この手の表記揺れはものすごい多いですよね。

ja_stop token filter

頻出語句で、検索に有用でない言葉を除去してくれます。

例）これ、それ、あれ、しかし、私、...etc

プラグイン内のリストがどこにあるかは分かりませんでしたが、一般にはこのようなリストになるようです。

kuromoji_number token filter

漢数字を直してくれます。

例）一〇〇〇（漢数字のゼロ）→　1000

ICUプラグイン機能のそれぞれ

ICU Normalization Character Filter

文字の正規化を行います。正規化について正確な仕様はこちらに記載されています

271　250　•••

半角・全角の統一、その他の記号類の統一のようです。

例）ｱﾊﾟ　→　アパート
例）ｱﾊﾟｰﾄ　→　アパート

上はかなり無理して一文字にしてましたね...

ICUはプログラムから使えるライブラリでもあります。例えばJava版ではJavadocはこちらです。

またICUの正規化の利用に関して、（プラグイン・ライブラリともに共通で）モードというのがあります。こちらは、上記仕様書の『Table 1. Normalization Forms』の4つのパターンです。デフォルトは一番下（４つ目）になっており、このままで問題ないみたいです。
パターンの違いに関してはこちらの方の記事が分かりやすいと思います。

ICU Tokenizer

これは今回使いませんでした。
辞書ベースで、タイ語、中国語、日本語、韓国語などのアジア言語をこの一つのTokenizerでサポートしているようです。それよりは、Kuromojiを使ったほうが良さそうであるのと、

> This functionality is experimental and may be changed or removed completely in a future release

まだ実験的な段階で、将来的に変更・除去される可能性があるそうです。

ICU Normalization Token Filter

こちらも使いませんでした。
説明を読むと、先ほどのICU Normalization Character Filterと同じ仕様のToken Filter版であるように見えるのと、ガイドでは

> You should probably prefer the Normalization character filter.

271　250　•••

ICU Folding Token Filter

これはぱっと読んで解釈ができませんでした😵ガイドによると

> Case folding of Unicode characters based on UTR#30

UTR#30に基づく表記変換のようです...仕様はこちらで、この仲を見ると、ひらがなとカタカナがある...HiraganaFoldingとはこちらに一覧があり、見ると

例）くう → クー
例）ふう → フー

というように、長音統一の模様...なるほど...

これは品詞として、何を検索に適用するかにも依存しそうですね。
名詞や動詞ではなかなか出てきそうにもないので、なんでしょう...擬音語でできている副詞みたいなものでしょうか？

例）ふうふう　→　フーフー

または感嘆詞とか...

例）あぁ！　→　アー！
例）おう！　→　オー！

入れるかどうかは任意ですね...（詳しい方がいらしたら、教えて頂けると助かります）
今回、私はこれらの品詞を使わなかったのと、副作用が分からなかったので、一旦入れませんでした。

ICU Collation Token Filter

Collations... ?

> Collations are used for sorting documents in a language-specific word order

言語の辞書順で、ソートをしてくれるようです。
英語だとアルファベット順、他の言語でもドイツ語のアルファベットみたいな文字

271　250　•••

が）でのソート...

解説では、電話番号フィールドに設定してありました。

今回は検索結果のスコア順に並んでほしいので、入れませんでした。

ICU Transform Token Filter

何を変換してくれるのか...と思ったら、transliteration（音訳）ができるそうです。

例）こんにちは → kon'nichiha.
例）你好 → ni hao.

...すごい...のですが...今回は入れませんでした。

Custom Analyzerの設定

やっと結論部です。先ほどの採用するモジュールをまとめて、カスタムアナライザにすれば完成です。
こちらのガイドに沿って検討しました。

適用順は？

Elasticsearchでは、
1. Character Filter（１文字ずつの処理）
2. Tokenizer（トークン化。単語区切りにする処理）
3. Token Filters（各トークンに対する処理）

の順に処理されます。適用順に上から記載します。

Character Filter	機能	Kuromoji Analyzerに組込み？
ICU Normalization Character Filter	文字の正規化	No
kuromoji_iteration_mark	踊り字の正規	No

271 250 •••

Tokenizer	機能	Kuromoji Analyzerに組込み？
kuromoji_tokenizer	トークン化	Yes

原形(baseform)を使う場合

Token Filters	機能	Kuromoji Analyzerに組込み？
kuromoji_baseform token filter	原形化	Yes
kuromoji_part_of_speech token filter	不要な品詞の除去	Yes
ja_stop token filter	ストップワードの除去	Yes
kuromoji_number token filter	漢数字の半角数字化	No(※1)
kuromoji_stemmer token filter	長音の除去	Yes(※2)

読み仮名(readingform)を使う場合

Token Filters	機能	Kuromoji Analyzerに組込み？
kuromoji_part_of_speech token filter	不要な品詞の除去	Yes
ja_stop token filter	ストップワードの除去	Yes
kuromoji_readingform token filter	読み仮名化	No(※1)(※2)
kuromoji_stemmer token filter	長音の除去	Yes(※2)

271 250 •••

(※1) kuromoji_numberとkuromoji_readingformの関係
この組み合わせは不思議な挙動をします。
kuromoji_number->kuromoji_readingformの順に使うと、全ての半角数字はtoken群から失われます。
kuromoji_readingformのみで使用すると、元々の半角数字はtoken群の中に現れます。
_analyzeコマンドの結果からは違いは読み取れないように感じました。

kuromoji_number token filterを使う目的は、漢数字を半角数字を置き換えたかったということですが、kuromoji_readingformを後から適用すると取得したかった半角数字消えてしまうとは…結局どちらかしか適用できませんでした。基準はどちらを優先するか、です。

- kuromoji_numberを適用しないか、先に読み仮名変換すれば「一二三」が「イチ、ニ、サン」のように一桁ずつバラバラの読み方で残ります。
- kuromoji_numberは「123」という一つのtokenにまとめてくれますので、「123」で検索した場合のみぴたりヒットします。

(※2)kuromoji_readingformとkuromoji_stemmerの関係
この組み合わせの順序は固定です。
kuromoji_readingform->kuromoji_stemmerの順だと正しく、長音が除去されます。
例）プリンター　→　プリンタ
逆にするとkuromoji_readingformからの出力は、元の「プリンター」に戻ります。
読み仮名はTokenの元の単語「プリンター」に基いて、出力しなおすためだと思われます。

実際の設定の記述は？

indexのsettingsは、こちらのガイドに沿って記述してみたところ、こんな感じになりました。

kibana(devtool)

```
PUT my_ja_map
{
  "settings": {
    "analysis": {
      "analyzer": {
```

271　250　•••

```
          "tokenizer": "kuromoji_tokenizer",
          "char_filter": [
            "icu_normalizer",
            "kuromoji_iteration_mark"
          ],
          "filter": [
            "kuromoji_baseform",
            "kuromoji_part_of_speech",
            "ja_stop",
            "kuromoji_number",
            "kuromoji_stemmer"
          ]
        }
      }
    }
  }
}
```

Token Filterの処理順は、基本的にはKuromoji Analyzerのデフォルトの順を尊重しました。

kuromoji_readingformは外してあります。読みかな検索をしたい場合、もう一つカスタムアナライザの設定を作成したいところですが、index一つにつき、カスタムアナライザは一つしか定義できないようです。kuromoji_readingformを使う場合は、kuromoji_baseformと交換してkuromoji_numberを外してください。

読み仮名にはどう対応する？

先述のとおり、読み仮名検索をすると、元の字体の検索は失われてしまいます。
元の文章のフィールドと、読み仮名のフィールドの２つを用意して同時に検索にかけると、広く検索をヒットさせた上で、読みと字体の両方ヒットしたほうがスコアが高いドキュメントとして取得できそうです。

さて。今回はここまで。専門家ではないので、とても苦労しました  ほんともう覚えきれないです。また、良くお使いの方で詳しい方がいらしたら、コメントにて教えて頂けると助かります。

各国語の設定（終わりに）

271 250 •••

います。たくさんありますね。

しかし、日本語だけでおなかいっぱいでした。

以上です。

新規登録して、もっと便利にQiitaを使ってみよう

1. **あなたにマッチした記事をお届けします**
2. **便利な情報をあとで効率的に読み返せます**

ログインすると使える機能について

新規登録

ログイン



@shin_hayata

フォロー

271

250

この記事は以下の記事からリンクされています

docker-compose.yml 一本で、日本語検索できるWiki.jsをサクッと構築する からリンク　8 months ago

Rails + ElasticSearch 調査メモ からリンク　3 years ago

[Elasticsearch] analyzerを使う前に把握しておきたい内容まとめ からリンク　3 years ago

【Elasticsearch×Java】Javaで取得されたクエリの実行結果が想定と異なったので調査→対応した内容まとめ からリンク　3 years ago

記事を読みながら理解する！Elasticsearch入門者が抑えておきたい内容まとめ からリンク　3 years ago

過去の9件を表示する

コメント

@hirokishirai (Hiroki Shirai)　2018-03-11 13:25　（編集済み）

271　250

一点ですが、

> kuromoji_readingformは外してあります。読みかな検索をしたい場合、もう一つカスタムアナライザの設定を作成したいところですが、index一つにつき、カスタムアナライザは一つしか定義できないようです。

は間違いかと思います。
カスタムアナライザは複数指定するとこができ、例えば、私は以下のようにインデックス時に使うアナライザ、検索時に使うアナライザで分けて定義したりしています。

```
{
  "analyzer": {
    "japanese_index_analyzer": {
      "type": "custom",
      "char_filter": [...],
      "tokenizer": "japanese_index_tokenizer",
      "filter": [...]
    },
    "japanese_search_analyzer": {
      "type": "custom",
      "char_filter": [...],
      "tokenizer": "japanese_search_tokenizer",
      "filter": [...]
    }
  }
}
```

1

あなたもコメントしてみませんか :)

新規登録

すでにアカウントを持っている方はログイン